

Índice y secuencia de trabajo

Orden de ejecución de los cuadernos:

- 01 · Creación de datos
- 02 · EDA de datos
- 03 · Preprocesamiento
- 04 · Baseline
- 05 · ML (CatBoost, LightGBM, RandomForest, XGBoost)
- 06 · DL (DeepAR, GRU, LSTM, N-BEATS. TFT y VanillaTransformer en GPU)
- 07 · Comparativa de modelos
- 08 · Análisis de Stock

Repositorio Github

<https://github.com/jmorala/TFMDS>

El repositorio es publico

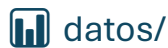
Estructura del Proyecto

 cuadernos/

Notebooks de Jupyter con el desarrollo completo del proyecto:

- **01_Creacion_datos.ipynb**: Creación y preparación inicial del dataset
- **02-EDA_datos.ipynb**: Análisis Exploratorio de Datos (EDA)
- **03_Preprocesamiento.ipynb**: Limpieza y transformación de datos
- **04_Baseline.ipynb**: Modelos baseline de referencia (Naive, Media Móvil)
- **05_ML_CatBoost.ipynb**: Modelo de Machine Learning con CatBoost
- **05_ML_LightGBM.ipynb**: Modelo de Machine Learning con LightGBM
- **05_ML_RF.ipynb**: Modelo de Machine Learning con Random Forest
- **05_ML_XGBoost.ipynb**: Modelo de Machine Learning con XGBoost
- **06_DL_DEEPAR.ipynb**: Modelo de Deep Learning DeepAR

- **06_DL_GRU.ipynb**: Modelo de Deep Learning con GRU (Gated Recurrent Unit)
- **06_DL_LSTM.ipynb**: Modelo de Deep Learning con LSTM
- **06_DL_NBEATS.ipynb**: Modelo de Deep Learning N-BEATS
- **06_DL_TFT_GPU.ipynb**: Modelo Temporal Fusion Transformer (GPU)
- **06_DL_VanillaTransformer_GPU.ipynb**: Transformer Vanilla (GPU)
- **07_Comparativa_modelos.ipynb**: Comparación exhaustiva de todos los modelos
- **08_Analisis_Stock.ipynb**: Análisis de costes de inventario y stock de seguridad



Datasets y archivos de resultados:

Datos originales:

- **Ventas.csv**: Datos históricos de ventas
- **Stock.csv**: Información de inventario
- **Calendario.csv**: Información de calendario y festivos
- **Promociones.csv**: Registro de promociones
- **DatosCicloAprovisionamiento.csv**: Ciclos de reaprovisionamiento por product
- **DatosPrecioMedio.csv**: Precios medios de productos

Datos procesados:

- **STDatosVentasTienda.csv**: Datos de ventas por tienda
- **df_train.csv**: Dataset de entrenamiento para modelos ML
- **df_test.csv**: Dataset de test para modelos ML
- **df_train_dl.csv**: Dataset de entrenamiento para modelos DL
- **df_train_dl_normalized.csv**: Dataset de entrenamiento normalizado para DL
- **df_test_dl.csv**: Dataset de test para modelos DL
- **df_test_dl_normalized.csv**: Dataset de test normalizado para DL
- **df_test_catboost.csv**: Predicciones del modelo CatBoost

Resultados de modelos:

- **resultados_metricas_baseline.csv**: Métricas de modelos baseline
- **resultados_metricas_catboost.csv**: Métricas del modelo CatBoost

- **resultados_metricas_lightgbm.csv**: Métricas del modelo LightGBM
- **resultados_metricas_rf.csv**: Métricas del modelo Random Forest
- **resultados_metricas_xgboost.csv**: Métricas del modelo XGBoost
- **resultados_metricas_deepar.csv**: Métricas del modelo DeepAR
- **resultados_metricas_GRU.csv**: Métricas del modelo GRU
- **resultados_metricas_lstm.csv**: Métricas del modelo LSTM
- **resultados_metricas_NBEATS.csv**: Métricas del modelo N-BEATS
- **resultados_metricas_TFT_gpu.csv**: Métricas del modelo TFT
- **resultados_metricas_transformer_gpu.csv**: Métricas del Transformer

[documentacion/](#)

Documentación técnica y referencias:

- **2016_yamazaki_StockSeguridad.pdf**: Paper de referencia para cálculo de stock de seguridad (Yamazaki, 2015)
- **EstimacionCostesStock.docx**: Documentación metodológica para estimación de costes

[lib/](#)

Librería de utilidades reutilizables:

- **__init__.py**: Inicialización del paquete Python
- **utils.py**: Funciones de utilidad general
- **metricas.py**: Funciones para cálculo de métricas de evaluación
- **graficos.py**: Funciones para visualización de modelos ML
- **graficos_dl.py**: Funciones para visualización de modelos DL
- **dl_utils.py**: Utilidades específicas para modelos de Deep Learning
- **EJEMPLO_IMPORT.txt**: Ejemplo de cómo importar las utilidades

Instalación

`pip install -r requirements.txt`

01 Creación de datos

Archivo_01_Creacion_datos.ipynb

se construye el dataset principal unificando las distintas fuentes facilitadas. Se han aplicado reglas de coherencia temporal para asegurar que las fechas de venta concuerdan entre orígenes; se eliminaron registros con fechas inconsistentes.

Unión de fuentes y normalización de columnas (nombres, tipos y formatos temporales).

Chequeos de integridad temporal: se eliminaron filas cuyas fechas no coincidían con la estructura temporal esperada.

Reconstruir ventas cuando stock 0: En el caso en que un producto-día el stock es cero, entonces las ventas serán 0, no porque no haya demanda si no porque no hay unidades disponibles para la venta. En este caso hay que reconstruir las ventas que se tendrían para ese producto-día. Se reconstruye con la media de N=7 días anteriores y posteriores ya que utiliza los datos más cercanos al día de rotura y así captura la tendencia temporal,

El caso de ventas = stock = 0 y bolOpen=0 no se considera rotura de stock, ya que la tienda no está abierta.

Si el día está cerrado y hay ventas se ponen las ventas a 0

Resultado: STDatosVentasTienda.csv

02 EDA de datos

Archivo: 02-EDA_datos.ipynb

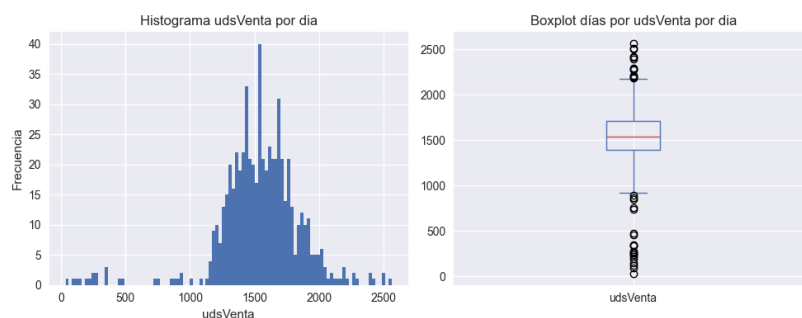
Análisis exploratorio para caracterizar las series de ventas, detectar outliers, estacionalidad y dependencia temporal. Se realizaron tests estadísticos que permitieron concluir que la serie es estacionaria.

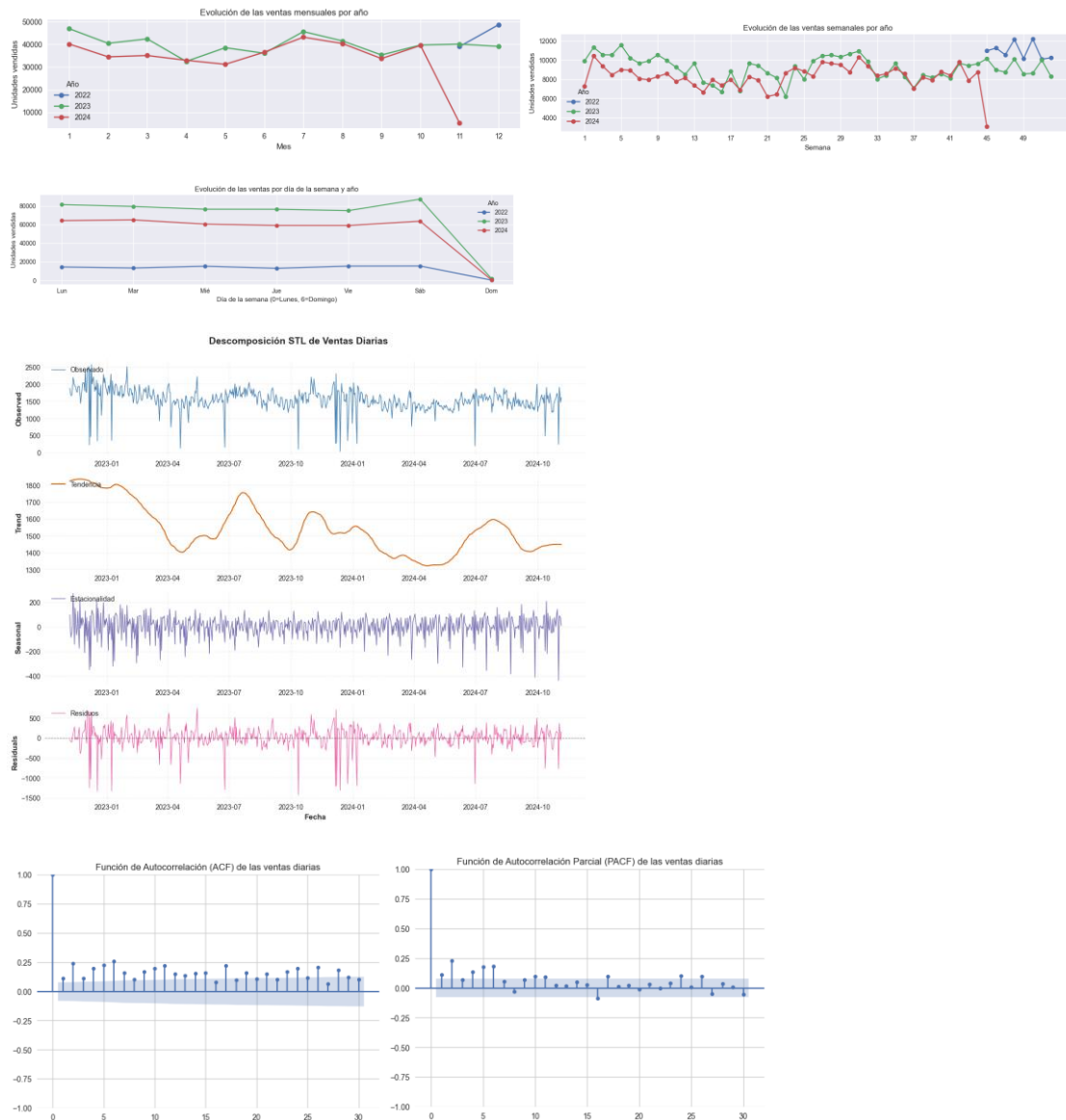
No se contemplan para el análisis los días en que la tienda está cerrada.

Principales análisis

- Análisis univariante por producto y categoría.
- Descomposición temporal (tendencia + estacionalidad + residuo).
- Chequeo de autocorrelación (ACF/PACF) y tests de estacionariedad.

Gráficos clave:





03 Preprocesamiento

Archivo: 03_Preprocesamiento.ipynb

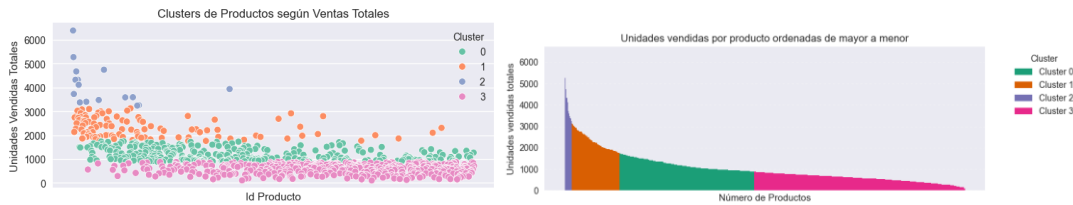
Preparación específica para dos familias de algoritmos — modelos de árboles (ej. CatBoost, LightGBM, XGBoost, RandomForest) y redes neuronales (ej. LSTM, GRU, DeepAR, N-BEATS, TFT, VanillaTransformer). Para cada familia se generó un juego de datos entreno/test adaptado.

Se incluyen características de calendario: día semana, mes, semana del año

Lag de ventas de los siete días anteriores: lag1 .. lag7

Se calcula un promedio en la que los valores recientes tienen un peso mayor que los valores pasados. Se calcula el EWMA a corto plazo (7 días) y a largo plazo (28 días), y con la diferencia se representa la tendencia de ventas del producto

Se determina clasificar los productos según sus ventas totales el 4 clusters:



Se definen los juegos de entrenamiento y de test:

- Registros en train: 625,800
- Registros en test: 27,714
- Fechas train: 2022-11-06 00:00:00 a 2024-10-05 00:00:00
- Fechas test: 2024-10-06 00:00:00 a 2024-11-05 00:00:00

Se realiza un encoding del código del producto para los algoritmos de árboles según la media de ventas de cada producto en el conjunto de entrenamiento:

- Productos únicos: 894
- Ejemplo de mapping (primeros 5 productos): {1: '8.69', 2: '7.30', 3: '5.09', 4: '2.97', 5: '3.83'}

Se guarda los juegos de entrenamiento y test para los algoritmos de árboles:

df_train y df_test, con las características que ya se disponían más las nuevas creadas. Columnas de los dataset: idSecuencia; producto; udsVenta; bolPromocion; bolOpen; bolHoliday; udsStock; rotura_stock; día_semana; mes; semana; lag_ventas_1; lag_ventas_2; lag_ventas_3; lag_ventas_4; lag_ventas_5; lag_ventas_6; lag_ventas_7; EWMA_corto; EWMA_largo; Tendencia_EWMA; Cluster; producto_encoded:

Para los algoritmos de redes neuronales se incluyen las siguientes características:

Para las características de calendario se utilizan las funciones de seno y coseno para dar sentido cíclico por lo tanto se crean las variables día_semana_sin, día_semana_cos, mes_sin_mes_cos, semana_sin, semana_cos.

Se normalizan las variables numéricas.

Se aplica codificación one-hot a la característica de cluster.

Se guardan los juegos de entrenamiento y test para los algoritmos de tipo

Deep learning (DL) df_train_dl.csv y df_test_dl.csv con las siguientes características: idSecuencia; producto; udsVenta; bolPromocion; bolOpen; bolHoliday; lag_ventas_1; lag_ventas_2; lag_ventas_3; lag_ventas_4; lag_ventas_5; lag_ventas_6; lag_ventas_7; EWMA_corto; EWMA_largo; Tendencia_EWMA; día_semana_sin; día_semana_cos; mes_sin; mes_cos; semana_sin; semana_cos; Cluster_0; Cluster_1; Cluster_2; Cluster_3.

Finalmente este juego de datos no contiene las variables numéricas normalizadas, ya que para los algoritmos de DL se utiliza la librería Neuralforecast que ya maneja internamente la normalización de los datos numéricos.

Resultado: df_train.csv, df_test.csv, df_train_dl.csv, df_test_dl.csv

Estructura recopilación métricas

En todos los algoritmos, se recopilan las métricas para analizar la precisión con la misma estructura, para así después analizar todos los algoritmos conjuntamente.

La estructura es la siguiente:

- algoritmo: Nombre del algoritmo/modelo (obligatorio)
- ndetalle: Nivel en que ha sido entrenado el modelo. Posibles valores: "Global", "Cluster", "Producto"
- cluster : Las métricas que presenta se han analizado específicamente para ese cluster. Posibles valores Nan, 0, 1, 2, 3
- producto : Las métricas que presenta se han analizado específicamente para ese producto. Posibles valores Nan o identificador del producto
- MAE
- MSE'
- RMSE'
- R2
- MAPE (%)
- SMAPE (%)
- RMSSE'
- MAE (%)

En cada algoritmo se analizan varios niveles de métricas y finalmente se guardan en ficheros tipo csv. El nombre del fichero tiene la estructura resultados_metricas_[Nombre algoritmo].csv

Para analizar por producto se han tomado los dos producto por cluster con más ventas en el dataset de entrenamiento.

04 Baseline

Archivo: 04_Baseline.ipynb

Definición y evaluación de baselines sencillos que sirven de referencia. Se implementan dos tipos:

- Naive: persistencia del último valor observado como predicción.
- Media7: media móvil de ventana 7 como predicción.

Los algoritmos se aplican a nivel de producto

El modelo generado se marca como NDetalle = 'Global', ya que no tiene sentido hacer un modelo a nivel cluster o producto porque daría el mismo resultado. Se toman las métricas a nivel global, por cluster y por producto

Resultado: resultados_metricas_baseline.csv

05 Algoritmos basados en árboles (ML)

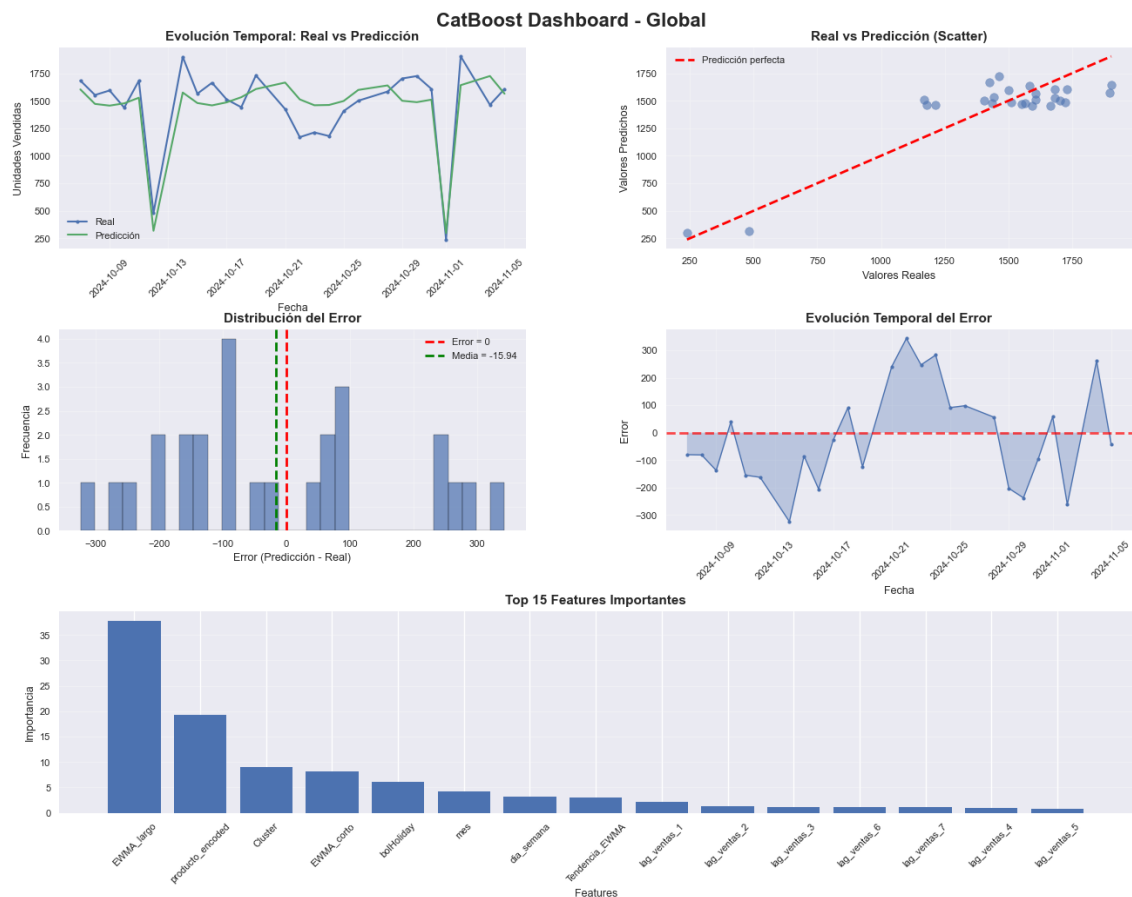
Archivos: 05_ML_CatBoost.ipynb, 05_ML_LightGBM.ipynb, 05_ML_RF.ipynb, 05_ML_XGBoost.ipynb

En cada archivo se entrenan modelos que implementan un algoritmo en concreto de árboles, se han seleccionado los que aparecían al analizar el estado del arte con mejor resultado. Para cada modelo se hace la predicción sobre los datos de tests y se toman las métricas y se guardan en ficheros csv.

Cada cuaderno ipynb tiene la misma estructura:

1. Se entrena un modelo NDetalle='Global' con todos los datos de entrenamiento, se utiliza la librería Optuna para probar varias combinaciones de hiperparámetros y quedarse con la que da mejor resultado.
2. Se toman las métricas del este modelo 'Global' para todas las predicciones, para cada cluster y para los dos productos top de cada cluster.
3. Se entrena un modelo para cada cluster NDetalle='Cluster' con los datos de entrenamiento del cluster en concreto, en estos casos los hiperparámetros que se han tomado son los mismos que se obtuvieron con Optuna para el caso global, ya que se asumen que serán iguales o muy parecidos y utilizar Optuna para cada modelo de cluster hubiera llevado demasiado tiempo.
4. Se toman las métricas del este modelo 'Cluster' para las predicciones para cada cluster y para los dos productos top de cada cluster.
5. Se entrenó un modelo NDetalle='Producto' para los dos productos top de cada cluster, se vuelven a utilizar como hiperparámetros los mismo que los del modelo Global.
6. Se toman las métricas del este modelo 'Producto' para las predicciones de los productos.

Para cada modelo se genera el siguiente dashboard:



Ejemplo de las métricas obtenidas:

Resumen de todas las métricas (todas las columnas):											
Algoritmo	NDetalle	Cluster	Producto	MAE	MSE	RMSE	R2	MAPE (%)	SMAPE (%)	RMSSE	MAE (%)
CatBoost	Cluster	3.0	314.0	0.1467	0.0559	0.2365	0.0000	NaN	200.00	NaN	NaN
CatBoost	Cluster	3.0	257.0	0.8122	1.3368	1.1562	-0.0075	79.59	187.57	0.7867	162.44
CatBoost	Cluster	3.0	NaN	1.0824	2.7236	1.6503	0.0874	55.84	169.25	0.7406	133.28
CatBoost	Cluster	0.0	413.0	1.7257	4.7045	2.1690	0.0280	33.40	102.08	0.7593	83.09
CatBoost	Cluster	0.0	NaN	1.8064	6.4825	2.5461	0.1193	39.84	125.77	0.7319	93.55
CatBoost	Cluster	0.0	294.0	2.2010	8.2678	2.8754	-0.0365	39.21	130.66	0.6625	119.22
CatBoost	Cluster	1.0	NaN	2.7718	12.8302	3.5819	0.1466	55.46	98.49	0.7159	72.69
CatBoost	Cluster	1.0	41.0	3.2815	18.3716	4.2862	0.0990	65.65	63.35	0.7316	49.60
CatBoost	Cluster	2.0	2.0	3.6435	20.3552	4.5117	0.0043	83.18	64.91	0.6269	57.76
CatBoost	Cluster	1.0	144.0	3.9010	24.5531	4.9551	0.0442	79.71	85.79	0.7406	69.00
CatBoost	Cluster	2.0	NaN	3.6127	25.0246	5.0025	0.2267	74.30	80.93	0.6957	62.91
CatBoost	Cluster	2.0	1.0	6.9381	112.6399	10.6132	0.0922	90.27	61.15	0.6682	58.76
CatBoost	Global	3.0	314.0	0.4125	0.2018	0.4492	0.0000	NaN	200.00	NaN	NaN
CatBoost	Global	3.0	257.0	0.8707	1.3328	1.1545	-0.0044	75.57	185.67	0.7855	174.13
CatBoost	Global	3.0	NaN	1.0803	2.7064	1.6451	0.0931	55.37	169.04	0.7382	133.02
CatBoost	Global	0.0	413.0	1.7067	4.6695	2.1609	0.0353	33.58	102.16	0.7565	82.17
CatBoost	Global	NaN	NaN	1.5676	5.5741	2.3609	0.2726	49.66	144.67	0.7235	96.01
CatBoost	Global	0.0	NaN	1.7990	6.4526	2.5402	0.1234	40.36	126.18	0.7302	93.17
CatBoost	Global	0.0	294.0	2.1824	8.4852	2.9129	-0.0638	37.83	130.57	0.6711	118.21
CatBoost	Global	1.0	NaN	2.7710	12.8129	3.5795	0.1478	55.19	98.58	0.7154	72.67
CatBoost	Global	1.0	41.0	3.1522	17.8270	4.2222	0.1257	60.14	62.00	0.7207	47.65
CatBoost	Global	2.0	2.0	3.5140	20.4862	4.5262	-0.0021	72.01	63.45	0.6289	55.71
CatBoost	Global	1.0	144.0	3.8136	24.9639	4.9964	0.0282	76.05	87.26	0.7468	67.45
CatBoost	Global	2.0	NaN	3.5895	25.5059	5.0503	0.2118	71.57	80.71	0.7023	62.50
CatBoost	Global	2.0	1.0	6.6507	120.3623	10.9710	0.0299	68.43	57.14	0.6907	56.33
CatBoost	Producto	3.0	314.0	0.2324	0.0990	0.3147	0.0000	NaN	200.00	NaN	NaN
CatBoost	Producto	3.0	257.0	1.4140	3.9064	1.9765	-1.9440	75.71	183.14	1.3448	282.81
CatBoost	Producto	0.0	413.0	1.8263	5.7153	2.3907	-0.1008	39.75	112.87	0.8369	87.93
CatBoost	Producto	0.0	294.0	2.6165	12.3356	3.5122	-0.5465	70.88	146.81	0.8092	141.73
CatBoost	Producto	1.0	144.0	3.6268	22.7240	4.7670	0.1154	63.82	82.13	0.7125	64.15
CatBoost	Producto	1.0	41.0	4.0972	26.1531	5.1140	-0.2826	79.28	80.67	0.8729	61.94
CatBoost	Producto	2.0	2.0	4.1124	27.7966	5.2722	-0.3597	78.59	69.28	0.7325	65.20
CatBoost	Producto	2.0	1.0	8.8232	148.9215	12.2033	-0.2002	125.32	76.65	0.7683	74.72

En el caso de 05_ML_CatBoost.ipyn, como el algoritmo CatBoost es el que mejor métricas ha sacado, se han guardado las predicciones del modelo global para utilizarlas en el cuaderno 08_Analisis_Stock.ipyn

Resultado: Para cada algoritmo se ha creado el fichero resultados_metricas_[Algoritmo].csv y además en el caso de CatBoost el fichero df_test_catboost.csv que contiene las predicciones por día del modelo Global de CatBoost.

06 Algoritmos basados en redes neuronales (DL)

Archivos: 06_DL_DEEPAR.ipynb, 06_DL_GRU.ipynb, 06_DL_LSTM.ipynb: Modelo de Deep Learning con LSTM, 06_DL_NBEATS.ipynb, 06_DL_TFT_GPU.ipynb, 06_DL_VanillaTransformer_GPU.ipynb

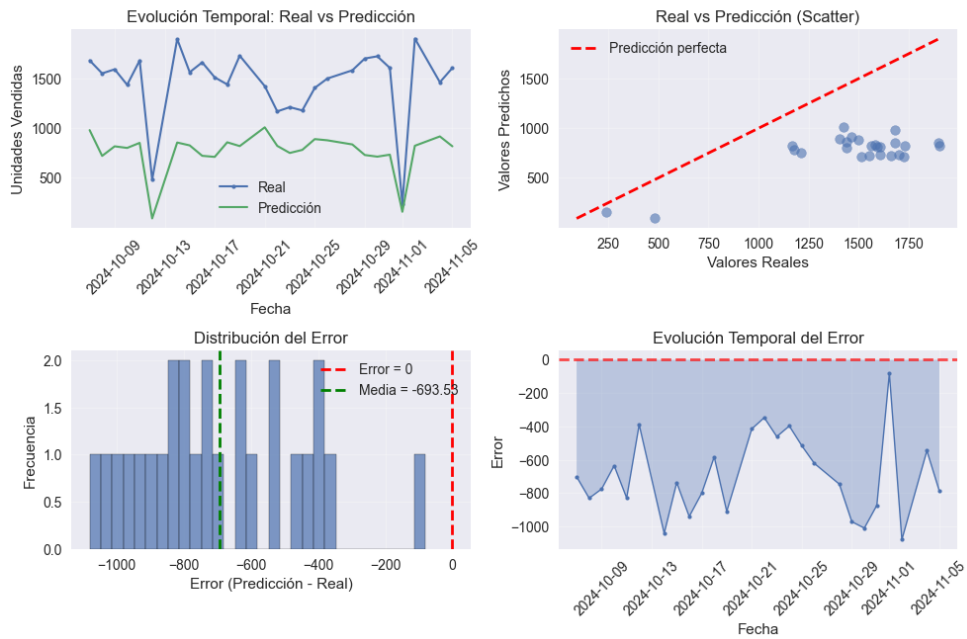
En cada archivo se entrenan modelos que implementan un algoritmo en concreto de redes neuronales, se han seleccionado los que aparecían al analizar el estado del arte con mejor resultado, y también los que se comentan en la última bibliografía obtenida (Deep Learning for Time Series Cookbook, Time Series Forecasting Using Generative AI : Leveraging AI for Precision Forecasting, Modern Time Series Forecasting with Python - Second Edition). Para cada modelo se hace la predicción sobre los datos de tests y se toman las métricas y se guardan en ficheros csv.

Cada cuaderno ipynb tiene la misma estructura:

1. Se entrena un modelo NDetalle='Global' con todos los datos de entrenamiento, en este caso no se ha podido utilizar la librería de Optuna ya que desbordaba los recursos del ordenador y el tiempo necesario.
2. Se toman las métricas del este modelo 'Global' para todas las predicciones, para cada cluster y para los dos productos top de cada cluster.
3. Se entrena un modelo para cada cluster NDetalle='Cluster' con los datos de entrenamiento del cluster en concreto, en estos casos los hiperparámetros que se han tomado son los mismos que para el caso global.
4. Se toman las métricas del este modelo 'Cluster' para las predicciones para cada cluster y para los dos productos top de cada cluster.
5. No se entrenó un modelo NDetalle='Producto' para los dos productos top de cada cluster, ya que no tiene sentido generar un modelo para cada artículo por el tiempo y recursos que se necesitan.

Para cada modelo se genera el siguiente dashboard:

LSTM Dashboard - Global



Ejemplo de las métricas obtenidas:

Resumen de todas las métricas (todas las columnas):											
Algoritmo	NDetalle	Cluster	Producto	MAE	MSE	RMSE	R2	MAPE (%)	SMAPE (%)	RMSSE	MAE (%)
LSTM	Cluster	3.0	314.0	0.0559	0.0054	0.0733	0.0000	NaN	200.00	NaN	NaN
LSTM	Cluster	3.0	257.0	0.5253	1.4883	1.2200	-0.1216	97.54	197.72	0.8301	105.06
LSTM	Cluster	3.0	NaN	0.8163	3.5454	1.8829	-0.1880	94.64	194.25	0.8449	100.51
LSTM	Cluster	0.0	413.0	1.7020	5.3761	2.3186	-0.1107	52.82	124.33	0.8117	81.95
LSTM	Cluster	0.0	NaN	1.7544	7.4312	2.7260	-0.0096	53.94	138.71	0.7836	90.86
LSTM	Cluster	0.0	294.0	1.9010	8.6282	2.9374	-0.0817	53.32	145.40	0.6767	102.97
LSTM	Cluster	1.0	NaN	2.7712	14.8582	3.8546	0.0117	50.37	103.40	0.7704	72.68
LSTM	Cluster	2.0	2.0	3.2375	18.7379	4.3287	0.0834	59.72	62.46	0.6014	51.33
LSTM	Cluster	1.0	144.0	3.7044	25.7434	5.0738	-0.0022	64.05	89.76	0.7584	65.52
LSTM	Cluster	1.0	41.0	3.8172	26.2516	5.1236	-0.2874	57.47	76.73	0.8746	57.70
LSTM	Cluster	2.0	NaN	3.6870	29.2432	5.4077	0.0963	68.67	79.13	0.7520	64.20
LSTM	Cluster	2.0	1.0	7.1810	152.5745	12.3521	-0.2297	60.98	70.30	0.7777	60.82
LSTM	Global	3.0	314.0	0.1908	0.0429	0.2071	0.0000	NaN	200.00	NaN	NaN
LSTM	Global	3.0	257.0	0.6211	1.4269	1.1945	-0.0754	91.00	193.58	0.8128	124.22
LSTM	Global	3.0	NaN	0.8473	3.3539	1.8314	-0.1238	90.82	188.97	0.8218	104.34
LSTM	Global	0.0	413.0	1.7149	5.5016	2.3456	-0.1366	54.81	126.66	0.8211	82.57
LSTM	Global	NaN	NaN	1.4428	6.5455	2.5584	0.1458	67.36	157.40	0.7840	88.36
LSTM	Global	0.0	NaN	1.7659	7.4613	2.7315	-0.0137	55.90	141.68	0.7852	91.46
LSTM	Global	0.0	294.0	1.9056	8.7809	2.9633	-0.1009	63.88	153.35	0.6827	103.22
LSTM	Global	1.0	NaN	2.8403	14.9759	3.8699	0.0039	51.03	105.49	0.7734	74.49
LSTM	Global	2.0	2.0	3.4648	19.8040	4.4502	0.0313	69.46	63.68	0.6183	54.93
LSTM	Global	1.0	144.0	3.8417	26.2420	5.1227	-0.0216	63.35	91.14	0.7657	67.95
LSTM	Global	2.0	NaN	3.6360	27.4192	5.2363	0.1527	66.73	80.73	0.7282	63.31
LSTM	Global	1.0	41.0	3.9651	27.8675	5.2790	-0.3667	57.94	79.13	0.9011	59.94
LSTM	Global	2.0	1.0	6.7735	139.9210	11.8288	-0.1277	54.42	56.86	0.7447	57.37

En el caso de 06_DL_TFT_GPU.ipyn y 06_DL_Vanillatransformer_GPU.ipynb se han preparado para que utilicen GPU y se han ejecutado en el entorno Google Colab que ofrece uso de GPU gratuita, aunque como también ofrece poca RAM se han tenido que bajar las dimensiones de los hiperparámetros relacionados con el tamaño de las capas

Resultado: Para cada algoritmo se ha creado el fichero resultados_métricas_[Algoritmo].

07 Comparativa de modelos

Archivo: 07_Comparativa_modelos.ipynb

Se leen todos los archivos que contiene métricas y se juntan en un único dataset para analizar las métricas.

Como métrica más importante se toma RMSE.

Para cada tipo de modelo:

- Se compara las métricas de los modelos Globales respecto el rendimiento en general, sobre cada cluster y sobre cada producto del top 2 de cada cluster
- Se compara las métricas de los modelos Cluster sobre cada cluster y cada producto top 2.
- Se compara las métricas de los modelos Producto, sobre el producto en concreto que se ha entrenado, en este caso los modelos tipo DL no tienen modelos implementados.

Para cada cluster y producto top 2:

- A nivel cluster se comparan todos los modelos generados, tipo Global y tipo Cluster para el cluster en concreto
- A nivel de producto se comparan todos los modelos generados, tipo Global, Cluster y Producto para el producto en concreto.

Se realizan varias comparativas adicionales, resultando el mejor CatBoost

Ejemplo de algunas gráficas:

En este cuaderno para los gráficos se ha utilizado la librería Plotly, Los gráficos no se ven bien en algunos visores.

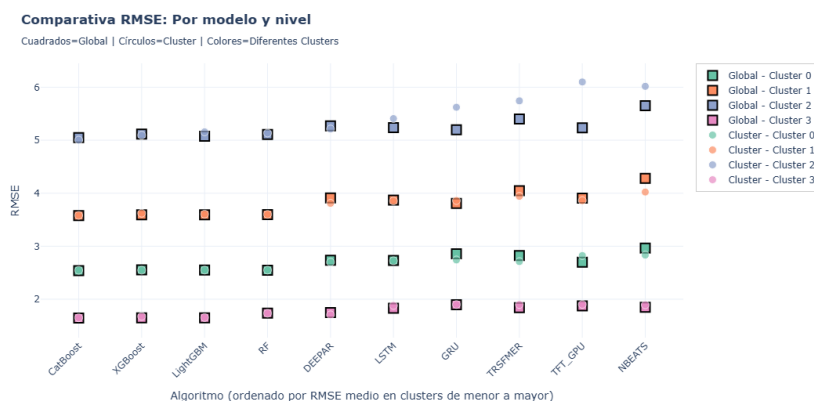




TABLA COMPARATIVA CONSOLIDADA - TOP ALGORITMOS

Algoritmo	MAE	RMSE	R2	MAPE (%)	SMAPE (%)	Ranking	RMSE
CatBoost	1.5676	2.3609	0.2726	49.6600	144.6700	1	
LightGBM	1.5641	2.3684	0.2680	49.7100	144.8200	2	
XGBoost	1.5662	2.3723	0.2656	49.8300	144.6400	3	
RF	1.5905	2.4020	0.2471	48.9200	143.9500	4	
DEEPAR	1.4573	2.5380	0.1594	63.3300	150.4600	5	
LSTM	1.4428	2.5584	0.1458	67.3600	157.4000	6	
TFT_GPU	1.4151	2.5697	0.1383	66.0500	163.0000	7	
GRU	1.4104	2.6169	0.1064	72.8600	162.2500	8	
TRSFMER	1.3974	2.6335	0.0950	69.2000	168.1600	9	

08 Análisis de Stock

Archivo: 08_Analisis_Stock.ipynb

Se implementa en este cuaderno el cálculo del stock de seguridad por medio del error de la previsión y el coste de stock por día.

Se utilizan los datos de predicción del algoritmo CatBoost (archivo df_test_catboost.csv) y se generan predicciones según método Naive media móvil de 7 días.

Las predicciones son sobre los últimos 30 días de los datos pasados, que quitando los días festivos son 26 días.

Los cálculos están implementados siguiendo el documento EstimacionCostesStock.docx.

Se realiza el cálculo de la mejora obtenida mediante CatBoost respecto al método Naive.

Se genera gráfica de evolución respecto cada escenario y tabla comparativa.

Gráficos relevantes:

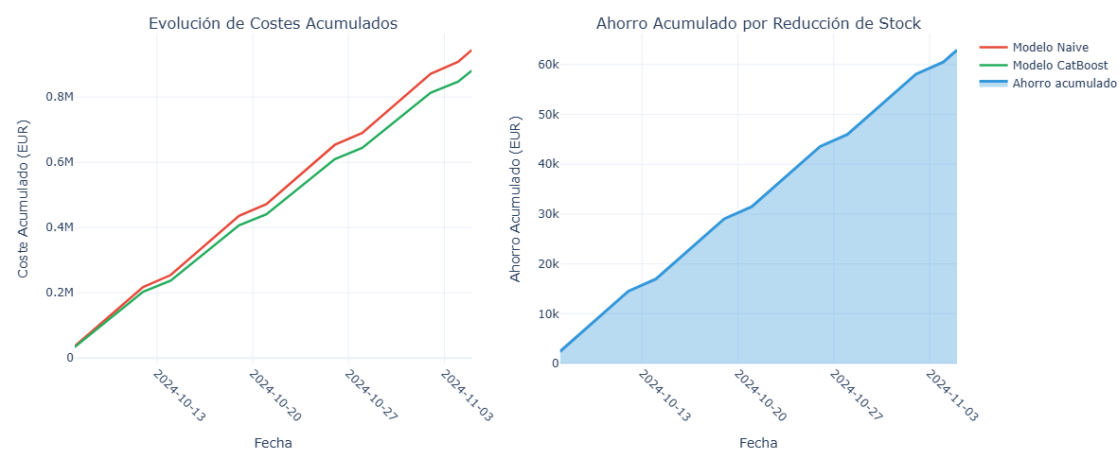


TABLA RESUMEN: COMPARATIVA MODELO NAIVE vs CATBOOST			
	Métrica	Modelo Naive	Modelo CatBoost
0	RMSE Promedio (unidades)	2.14	2.03
1	Stock Seguridad Promedio (unidad)	15.56	14.72
2	Valor Stock Promedio (€)	812.36	758.23
3	Coste Diario Promedio (€)	40.62	37.91
4	Coste Total Periodo (€)	944,124.06	881,212.93
5	Ahorro Total Periodo (€)	-	62,911.13
6	Ahorro Anual Proyectado (€)	-	883,175.45
7	Reducción Stock (%)	-	5.01%
8	Reducción Costes (%)	-	6.66%

9 rows x 3 cols

10 per page

Page 1 of 1